

6.3D and 6.4HD: What Worked, What Failed, and Why

A detailed comparison of two smart-lighting architectures, written as a learning reference for redesigning the system as a reliable TypeScript air-conditioning controller.

PURPOSE

Understand the architecture through failure scenarios, not technology names. Learn how state ownership, delivery semantics, atomicity, idempotency, and scaling determine correctness.

Prepared 1 October 2026 | Australia/Sydney

Contents

Contents	2
1. Executive comparison	4
1.1 Vocabulary used throughout	5
2. 6.3D: serverless AWS architecture	6
2.1 What it achieved	6
2.2 Implemented event path	6
2.3 Problem: incompatible sensor contract	6
2.4 Problem: process-local current state	7
2.5 Problem: cache updated before downstream success	7
2.6 Problem: no enforceable event idempotency	7
2.7 Problem: non-atomic DynamoDB histories	8
2.8 Problem: incomplete input history and weak reproducibility	8
3. 6.4HD: containerised microservices architecture	9
3.1 What it achieved	9
3.2 Implemented automatic event path	9
3.3 Problem: ordinary MQTT subscriptions on every replica	9
3.4 Problem: concurrent light-state transitions	10
3.5 Problem: state and command writes are still non-atomic	10
3.6 Problem: automation-rule configuration diverges between pods	11
3.7 Problem: different entry points produce different workflows	11
3.8 Problem: autoscaling amplifies an incorrect delivery model	11
3.9 Operational complexity without full independence	11
4. Failure scenarios side by side	13
4.1 Why a correct final state is not enough	13
4.2 Why retries are normal, not exceptional	13
5. Corrective engineering patterns	14
5.1 Explicit event contract and validation	14
5.2 Stable identity and idempotent persistence	14
5.3 Atomic state transition	14
5.4 Separate desired, commanded, and applied state	14
5.5 Correct replica work distribution	14
5.6 Shared versioned configuration	14
5.7 Reproducible infrastructure	14
5.8 Failure-focused test matrix	15
6. Lessons for the air-conditioning rework	16
6.1 Recommended month-one boundary	16
6.2 Keep, adapt, or reject	16
6.3 Proposed state model	16
6.4 A thin first vertical slice	16

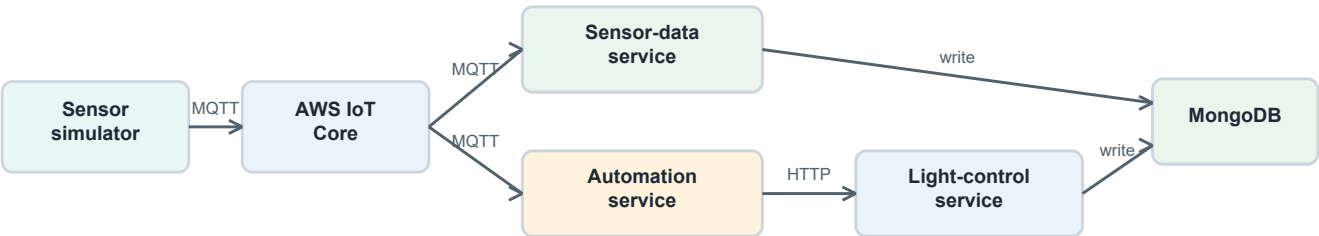
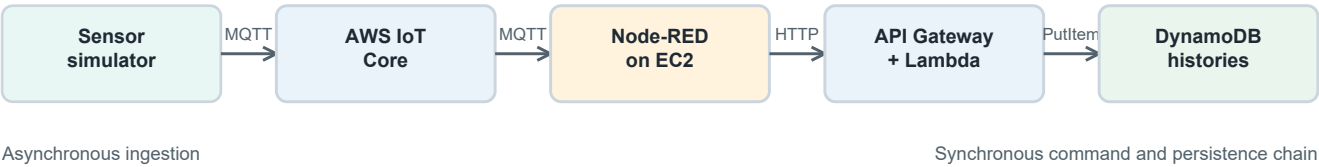
7. Review questions	18
7.1 Compact answer key	18
8. Evidence and limitations	19
8.1 Principal retained evidence	19
8.2 Claims this guide intentionally does not make	19

How to use this guide

Read the architecture overview first, then study each failure as a chain: assumption -> implementation -> triggering event -> incorrect result -> control. The goal is to be able to explain why a design is correct, not merely remember a preferred technology.

1. Executive comparison

Both projects achieved a real end-to-end IoT flow. The main difference is not that one used AWS serverless services and the other used Kubernetes. The deeper difference is where responsibilities and state were placed. 6.3D concentrated decisions in Node-RED and retained previous state in local memory. 6.4HD separated telemetry storage, decision calculation, and light-state management into services, then moved current light state into MongoDB. That change improved durability, but the replicated MQTT and database workflows introduced new concurrency and duplicate-processing risks.



Concern	6.3D	6.4HD	Main lesson
Ingestion	MQTT into Node-RED	MQTT fan-out to storage and automation	Different responsibilities may fan out; replicas of one responsibility should distribute work.
Current state	Process-local Node-RED map	Shared MongoDB LightState	Correctness-critical state needs an explicit durable owner.
Persistence	Two sequential DynamoDB writes	Sensor, current-state, and command collections	Related writes need atomicity; repeated inputs need idempotency.
Scaling	EC2/Node-RED plus Lambda scaling	Kubernetes HPA and Services	Replica count must not multiply business effects.
Reproducibility	Important deployment state remained in AWS	More manifests retained, but AWS certificates and runtime dependencies remained	Version the full runnable system and verification procedure.

Senior-engineering conclusion

6.4HD was not simply better because it used microservices and Kubernetes. It improved some state and deployment concerns while creating distributed-system failure modes that the implementation did not fully control.

1.1 Vocabulary used throughout

Term	Meaning in this guide
Event	A record that something happened, such as a sensor measuring 27 degrees or detecting motion.
Ingestion	The boundary where an external event is received, parsed, validated, and admitted for processing.
State	Information whose current value affects future decisions, such as the current light mode or air-conditioner mode.
Idempotency	Repeating the same logical operation produces no additional business effect.
Atomicity	A group of related changes either all commit or none commit.
Replica	One running copy of a service. Several replicas should improve capacity or availability without changing business meaning.
Shared subscription	A group of MQTT subscribers where one group member is selected for a publication instead of every member receiving it.

2. 6.3D: serverless AWS architecture

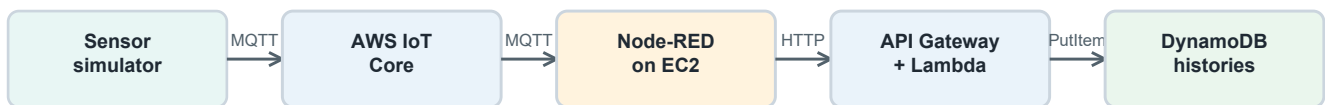
2.1 What it achieved

- Created an end-to-end path from simulated IoT readings to automated lighting decisions and durable history records.
- Used MQTT through AWS IoT Core to decouple sensor publishers from the Node-RED consumer.
- Encoded understandable lighting rules based on motion, light intensity, and time of day.
- Suppressed unchanged commands by comparing a newly calculated state with the previous calculated state.
- Exposed the command path through API Gateway and Lambda and persisted command and state histories in DynamoDB.
- Demonstrated cloud deployment components, monitoring, and scaling concepts rather than remaining a local-only prototype.

What deserves credit

The system was not valueless because it contained flaws. It proved integration across devices, MQTT, rule processing, HTTP, serverless compute, and persistence. The rework is valuable because it can preserve those ideas while correcting their reliability model.

2.2 Implemented event path



Asynchronous ingestion

Synchronous command and persistence chain

The sensor side was asynchronous: the simulator published without waiting for the complete automation and persistence workflow. After Node-RED produced a command, the remaining path behaved synchronously: Node-RED waited for the API request, API Gateway waited for Lambda, and Lambda awaited each DynamoDB operation before returning. JavaScript `async/await` did not make that service chain architecturally asynchronous.

2.3 Problem: incompatible sensor contract

The standalone sensor simulator published to `test/topic` and used the field `ambientLight`. The retained decision flow subscribed to `iot/sensors/+/+/+` and expected `lightId`, `light_intensity`, and `motion_detected`. The state-validation and load-test programs used the latter contract.

Stage	Expected	Possible actual result
Topic routing	Simulator topic matches Node-RED filter	The publication never reaches the retained decision flow.
Field parsing	Required fields are present and correctly typed	Missing values become zero, false, or undefined instead of causing rejection.
Decision	Decision reflects the real sensor reading	Malformed input can look like a valid dark/no-motion reading.

How to combat it

Define one versioned event contract. Validate topic, required fields, types, ranges, identifiers, and timestamps at the ingestion boundary. Run a contract test that publishes an event from the real simulator and proves that the real consumer accepts it.

2.4 Problem: process-local current state

Node-RED stored previous calculated states in `global.lightStates`, an in-memory map local to one runtime. DynamoDB contained durable history, but Node-RED did not rebuild its map from that history and did not share the map with other instances.

Failure example: restart

```
Before restart: cache[light-A] = ON
Node-RED restarts: cache = empty
Next reading calculates ON
No previous state exists -> Node-RED emits another ON command
```

The output looks like a legitimate state change even though the physical or simulated light was already on. The event history gains a redundant command and the first post-restart event behaves differently from the same event before restart.

Failure example: two replicas

```
Reading 1 for light-A -> instance N1 -> cache N1 becomes ON
Reading 2 for light-A -> instance N2 -> cache N2 has no entry
N2 treats ON as a new state and emits another command
```

How to combat it

Use one explicit source of truth for correctness-critical state, recover state before processing, or partition events so that all events for an entity reach a state owner. A cache may improve performance, but it must not silently become the only truth.

2.5 Problem: cache updated before downstream success

The Node-RED decision function updated `global.lightStates` before its output reached the downstream HTTP Request node. The cache therefore represented an intended command, not a confirmed command or applied device state.

1. Calculate desired state = ON
2. Update local cache to ON
3. HTTP request to API Gateway fails
4. Next reading also calculates ON
5. Cache says ON, so the retry opportunity is suppressed

Incorrect result

The system may believe the transition was handled even though Lambda and DynamoDB never received it. A later identical reading does not repair the failure because state-change suppression treats it as unchanged.

How to combat it

Model desired state, issued command, and confirmed applied state separately. Do not mark an operation complete before the required effect is confirmed. Use durable retries or an outbox when a database change and message publication must be coordinated.

2.6 Problem: no enforceable event idempotency

State-change suppression answered 'is the calculated mode different?' It did not answer 'have I already processed this exact event or command?' No stable producer-generated event identifier and database uniqueness rule was evident across the complete path.

```
Logical command C1 reaches Lambda and is written.
The HTTP response is lost.
The caller retries C1.
Without a stable idempotency key, the retry can look like a new command.
```

How to combat it

Generate a stable eventId or commandId once at the producer and preserve it across retries. Enforce uniqueness in the database. If the identifier already exists, return the previously recorded result without repeating the actuator or history effect.

2.7 Problem: non-atomic DynamoDB histories

Lambda awaited one PutItem for command history and then another PutItem for light-state history. Waiting sequentially established order; it did not create a transaction.

```
Put LightCommands succeeds
Put LightStates fails after retries
Lambda returns an error
Database now contains command history without matching state history
```

How to combat it

If both records form one invariant, write them in one database transaction. If eventual consistency is acceptable, explicitly model the intermediate state and provide a repair process. Never assume two awaited writes are atomic.

2.8 Problem: incomplete input history and weak reproducibility

- Readings that produced no command were not independently persisted, so historical non-decisions could not be reconstructed or replayed.
- The report described a shared MQTT subscription while the exported Node-RED flow contained an ordinary subscription. The exact deployed configuration is therefore uncertain.
- The original AWS account is unavailable, so live policies, deployed versions, QoS/session settings, and runtime resources can no longer be inspected.

How to combat it

Version application code, infrastructure, schemas, policies, runtime configuration, and verification instructions together. Decide explicitly whether raw-event history is required for audit, replay, debugging, or algorithm evaluation.

3. 6.4HD: containerised microservices architecture

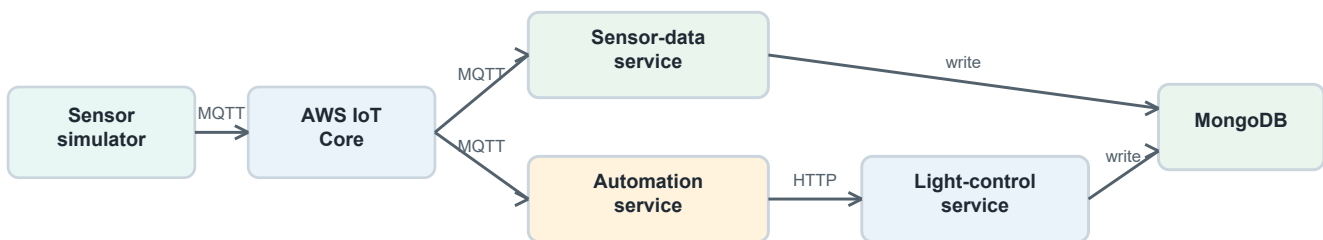
3.1 What it achieved

- Separated telemetry persistence, automation decisions, current-state management, and external HTTP access into named services.
- Persisted every received sensor reading in MongoDB rather than only retaining inputs attached to emitted commands.
- Moved current light state from a Node-RED process cache into a shared durable LightState collection.
- Added containers, Kubernetes Deployments and Services, health probes, resource requests and limits, and Horizontal Pod Autoscalers.
- Allowed independent HTTP routing through a custom Express API gateway and internal ClusterIP services.
- Retained command history while making current state directly queryable.

Real improvement

Moving current light state into shared durable storage directly addressed restart loss and per-instance cache divergence. It did not, however, make concurrent read-check-update workflows atomic.

3.2 Implemented automatic event path



The external Express API gateway is a separate HTTP facade and is not in this automatic MQTT path.

The sensor-data service did not forward readings to automation. Both services independently subscribed to AWS IoT Core. The sensor-data branch stored telemetry; the automation branch calculated a desired state and synchronously called the light-control service. The Express API gateway was a separate client-facing HTTP facade, not part of this automatic path.

3.3 Problem: ordinary MQTT subscriptions on every replica

The Kubernetes manifests configured three sensor-data replicas and three automation replicas. Each pod created its own MQTT connection and subscribed normally to the same topic filters. Kubernetes Services could load-balance inbound HTTP, but they were not between AWS IoT Core and those outbound MQTT connections.

```

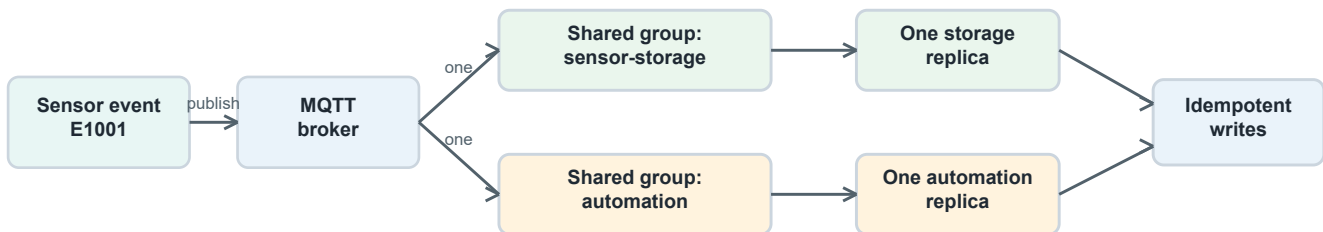
One event E1001
-> sensor-data S1 inserts E1001
-> sensor-data S2 inserts E1001
-> sensor-data S3 inserts E1001
-> automation A1 calculates ON
-> automation A2 calculates ON
-> automation A3 calculates ON
  
```

Replicas per service	Ordinary subscription: storage attempts	Ordinary subscription: automation executions	Separate shared groups
1	1	1	1 storage + 1 automation
3	3	3	1 storage + 1 automation
10	10	10	1 storage + 1 automation

Why this violates scaling semantics

Replica count should affect capacity and availability, not the number of logical records or device commands produced from one event. In the retained implementation, adding pods changes business-effect count.

Correct grouping



```

Sensor-data replicas: $share/sensor-storage/iot/sensors/+/+/+
Automation replicas: $share/automation/iot/sensors/+/+/+
  
```

The groups require different names. If both service types joined one shared group, they would compete with each other: some events would only be stored, while others would only be automated. Separate groups preserve intentional fan-out between responsibilities while distributing work within each responsibility.

Shared subscription is not exactly-once

A crash, reconnect, QoS redelivery, or producer retry can still repeat a logical event. Shared subscriptions control which replica receives a delivery; stable identifiers and idempotency control what repeated delivery is allowed to do.

3.4 Problem: concurrent light-state transitions

The light-control endpoint used a read-check-write sequence: read LightState, decide whether desiredState differs, update LightState, then insert LightCommand. Multiple HTTP requests could execute this sequence concurrently against shared MongoDB.

```

Initial state: OFF
R1 reads OFF
R2 reads OFF
R1 concludes OFF -> ON
R2 concludes OFF -> ON
R1 updates state and inserts command
R2 updates state and may insert another command
  
```

The unique lightId prevents two current-state documents, but it does not by itself prevent duplicate command-history entries. A final state of ON can therefore hide an incorrect history containing two OFF-to-ON commands.

How to combat it

Use an atomic conditional update or transaction. Include the expected previous version in the update, or use an optimistic concurrency version. Give the source event a stable identifier and enforce command uniqueness so a repeated event cannot insert a second command.

3.5 Problem: state and command writes are still non-atomic

```

Update LightState succeeds
Insert LightCommand fails
HTTP request returns 500
Current state changed, but command history is missing
  
```

6.4HD changed the storage technology and data model but retained the same fundamental partial-write risk: two related database operations were not one transaction.

General lesson

Changing DynamoDB to MongoDB does not fix atomicity. Correctness follows the transaction boundary and invariant, not the database brand.

3.6 Problem: automation-rule configuration diverges between pods

Each automation pod created an in-memory AUTOMATION_RULES object. The HTTP PUT endpoint modified only the pod that received the request. The automation service connected to MongoDB, but the retained implementation did not persist or reload the rules.

```
Initial: A1=200, A2=200, A3=200
PUT /rules is routed to A2
After update: A1=200, A2=250, A3=200
The same 225-lux event can produce different decisions depending on the pod
```

How to combat it

Treat decision configuration as shared versioned state. Store it centrally, distribute immutable configuration during deployment, or broadcast a versioned update and verify adoption. A health endpoint should report the active rules version.

3.7 Problem: different entry points produce different workflows

Entry point	What happens	What does not happen
MQTT publication	Sensor reading is stored and automation is evaluated through independent subscriptions	The external API gateway is not used
POST /api/sensors/readings	Sensor-data service stores the reading	Automation is not triggered
POST /api/automation/process	Automation evaluates the supplied reading and calls light-control	The reading is not necessarily stored by sensor-data
POST /api/lights/state	Light-control receives a desired state directly	Automation rules are bypassed

How to combat it

Define the contract of every entry point. If two entry points represent the same business operation, route them through the same application use case. If they intentionally differ, name and document them so callers cannot assume equivalent effects.

3.8 Problem: autoscaling amplifies an incorrect delivery model

The Horizontal Pod Autoscalers reacted to CPU and memory utilisation. Those metrics can indicate resource pressure, but they do not reveal whether messages are duplicated, rules disagree, database transitions race, or commands are repeated.

```
Higher event rate -> higher CPU -> more MQTT-consuming pods
More pods -> more ordinary subscriptions
More subscriptions -> more processing attempts per event
More attempts -> more MongoDB and light-control load
```

How to combat it

First prove the invariant 'one logical event creates one effect per responsibility' across several replica counts. Then scale using workload-relevant metrics such as queue lag, event-processing latency, error rate, and database contention, alongside CPU and memory.

3.9 Operational complexity without full independence

- Four Node.js services required separate images, Deployments, Services, probes, resource limits, HPAs, environment variables, secrets, and troubleshooting paths.
- The services communicated over a network and could fail independently, introducing distributed-system latency and partial-failure concerns.
- The services shared one MongoDB database URI, reducing data ownership independence while retaining microservice operational cost.
- AWS IoT certificates and external cloud configuration remained required, so the system was not fully local or self-contained.
- The architecture provided useful Kubernetes practice, but the application requirements did not clearly require this amount of distribution.

How to combat it

Begin with a modular monolith when one developer owns the system and independent deployment is not required. Preserve module boundaries in code. Extract a service only when scaling, ownership, security, availability, or deployment requirements justify the network boundary.

4. Failure scenarios side by side

The following scenarios demonstrate why an architecture can return plausible final state while still producing incorrect histories, missed effects, or non-reproducible behaviour.

Trigger	6.3D result	6.4HD result	Required control
Restart	Node-RED loses previous state and emits a redundant first command	MongoDB current state survives, but process-local rule changes disappear	Durable domain state and versioned configuration
Scale from 1 to 3	Per-instance Node-RED caches may diverge; subscription behaviour is uncertain	Every ordinary MQTT subscriber processes the event, multiplying inserts and automation	Correct work distribution plus replica-independent invariants
HTTP timeout after success	Caller may retry because it cannot know whether Lambda completed	Automation caller may retry light-control after an uncertain response	Stable idempotency key and stored outcome
Second database write fails	Command history can exist without state history	Current LightState can change without LightCommand history	Database transaction or explicit repairable workflow
Two requests race	Process-local state can produce conflicting views across instances	Both requests can read the same old state and log the same transition	Atomic conditional update, versioning, and command uniqueness
Malformed sensor payload	Missing fields silently become defaults or undefined values	Flexible fields and fallback logic can also hide contract errors	Runtime schema validation and rejection telemetry

4.1 Why a correct final state is not enough

Two concurrent requests both write ON.
 Final LightState = ON.
 A simple test passes.
 Command history contains two OFF -> ON commands.
 Energy metrics, audits, and physical actuation may now be wrong.

A robust test asserts the invariant and side effects, not only the final row. For one source event, check the final state, command count, sourceEventId uniqueness, response, and logs.

4.2 Why retries are normal, not exceptional

Networks can fail after the receiver completes work but before the sender receives confirmation. The sender cannot distinguish 'the server did nothing' from 'the server succeeded but the response was lost.' Retrying is therefore normal. Idempotency is what makes a retry safe.

Mental model

At-least-once delivery means the event is not silently abandoned, but it may be processed repeatedly. Exactly-once business effect is achieved through application and database design, not by assuming the transport will never duplicate.

5. Corrective engineering patterns

5.1 Explicit event contract and validation

Define required fields, types, units, ranges, identifiers, and timestamps. Reject malformed events before rule evaluation. Keep producer and consumer contract tests in the repository.

```
{"eventId":"sensor-7:18452","deviceId":"sensor-7","measuredAt":"2026-10-01T08:30:00Z","temperatureC":27.2,"sequenceNumber":18452}
```

5.2 Stable identity and idempotent persistence

Generate the identifier once at the producer and reuse it for every retry. Enforce a unique index such as UNIQUE(eventId) for readings and UNIQUE(sourceEventId, targetId, commandType) for derived commands.

```
First E1001 -> insert and apply  
Repeated E1001 -> return previously recorded outcome; no second effect
```

5.3 Atomic state transition

Place the state update, command creation, and processed-event marker inside one transaction when they form one invariant. Alternatively use an atomic conditional update with an expected version and retry on conflict.

```
BEGIN  
claim eventId E1001  
update state where version = 7  
insert command sourced from E1001  
COMMIT
```

5.4 Separate desired, commanded, and applied state

A rule produces desired state. A command records an attempt. Applied state records acknowledgement from the actuator. Treating these as one value hides downstream failure.

```
desired=COOL -> command=PENDING -> actuator ack -> applied=COOL
```

5.5 Correct replica work distribution

Use one shared group per service responsibility when a single delivery should be handled by one replica. Preserve separate groups for independent responsibilities. Do not mistake shared delivery for idempotency or per-device ordering.

```
$share/telemetry-storage/home/+/room/+/temperature  
$share/aircon-automation/home/+/room/+/temperature
```

5.6 Shared versioned configuration

Decision thresholds must be consistent across replicas. Store one configuration version, load it predictably, and include the version in decisions and diagnostics.

```
rulesVersion=12; coolStartAboveC=26; heatStartBelowC=19
```

5.7 Reproducible infrastructure

A fresh developer should be able to start the system from repository artifacts, run a smoke test, and understand required secrets without access to the original cloud account.

```
clone -> configure documented secrets -> docker compose up -> run integration test
```

5.8 Failure-focused test matrix

Test	Expected invariant
Publish the same eventId twice	One reading and at most one derived command exist.
Crash after database commit but before acknowledgement	Redelivery returns the stored outcome without repeating the effect.
Run three consumer replicas	Business-effect count remains the same as with one replica.
Send two state-changing requests concurrently	Only one valid transition from the expected version commits.
Fail command-history insertion	The current-state change is rolled back or an explicit repairable intermediate state remains.
Restart the application	Current state and active rule version remain consistent.
Send malformed temperature, missing ID, or stale sequence	The event is rejected or quarantined and does not alter device state.

6. Lessons for the air-conditioning rework

The rework should not recreate either legacy system component for component. Its purpose is to demonstrate that the same class of event-driven system can be built with explicit contracts, state ownership, failure behaviour, and tests.

6.1 Recommended month-one boundary

```
Temperature simulator -> Mosquitto -> NestJS application -> PostgreSQL
-> simulated air-conditioner command
```

A modular NestJS application is sufficient for the first milestone. It can retain clear modules for ingestion, automation, devices, and persistence without creating independent deployments before they are justified.

6.2 Keep, adapt, or reject

Legacy idea	Decision	Reason
MQTT sensor ingestion	Keep	It models device-to-backend communication well and decouples publishers from consumers.
Separate automation logic	Keep as a module	Pure decision logic is easy to test; it does not yet require a separate deployable service.
Persistent command history	Keep	It supports audit, debugging, and failure analysis when linked to source event IDs.
Process-local current state	Reject	Restart and replica correctness must not depend on one process cache.
Raw sensor history	Adapt	Retain readings needed for debugging and replay with explicit retention rules.
Multiple microservices	Defer	One developer and one month do not yet justify independent deployment and distributed failure cost.
Kubernetes autoscaling	Reject for month one	Correctness and realistic load requirements must exist before orchestration complexity.
Stable IDs and idempotency	Add from the beginning	They are easiest to design before data and handlers depend on unsafe assumptions.
Atomic state plus command write	Add from the beginning	The controller needs one consistent record of transitions.

6.3 Proposed state model

State	Meaning	Owner
TemperatureReading	Immutable measurement received from a sensor	Ingestion/persistence module
DesiredMode	OFF, COOL, or HEAT calculated from validated inputs and rules	Automation decision module
AirconState	Current confirmed or simulated operating mode	Device/state module
AirconCommand	A requested transition linked to a source reading and previous state	Device/state module
ProcessedEvent	Proof that a stable eventId has already produced its permitted effects	Transactional application boundary
AutomationRules	Thresholds and rule version used to calculate decisions	Versioned configuration owner

6.4 A thin first vertical slice

- Accept one valid temperature event with a stable eventId.
- Reject one invalid event using runtime schema validation.
- Persist the valid reading once.

- Calculate one deterministic OFF/COOL/HEAT decision using a pure function.
- Atomically update current state and record a command only when a transition is required.
- Process the same eventId twice and prove that the second attempt creates no additional effect.
- Restart the application and prove that the persisted state remains correct.

Definition of progress

The rework becomes resume-worthy through explainable correctness, tests, and engineering decisions, not through the number of technologies or infrastructure components it contains.

7. Review questions

- 1 Why did the 6.3D first event after restart behave differently from the same event before restart?
- 2 Why did awaiting two DynamoDB writes not make them atomic?
- 3 Why was 6.4HD MQTT fan-out valid between service types but incorrect between replicas of one service type?
- 4 Why must sensor-storage and automation use different shared-group names?
- 5 Why does a shared subscription not remove the need for idempotency?
- 6 How could two light-control requests both record OFF-to-ON even if the final LightState is correct?
- 7 Why can an in-memory automation-rule update produce inconsistent decisions in Kubernetes?
- 8 What is the difference between state-change suppression and duplicate-event handling?
- 9 Which related air-conditioning records should be written atomically, and what invariant do they protect?
- 10 What evidence would justify splitting the month-one NestJS application into separate services later?

7.1 Compact answer key

#	Answer
1	The in-memory previous-state map was empty after restart, so the first calculated state looked new.
2	Sequential completion provides order, not all-or-nothing commit behaviour.
3	Storage and automation have different responsibilities and each needs the event; replicas of one responsibility should share its workload.
4	One shared group would make storage and automation compete, causing one responsibility to be skipped per delivery.
5	Failures and retries can redeliver a logical event even when only one group member receives each delivery.
6	Both requests can read OFF before either commits ON, then both conclude that a transition is required.
7	The update reaches one pod's memory; other pods retain older values and restarts reload defaults.
8	State-change suppression asks whether desired state differs; idempotency asks whether this logical event has already produced its effect.
9	Reading claim, state transition, processed-event marker, and command record should share a transaction when the use case requires one consistent transition.
10	Independent deployment, scaling, ownership, security, or availability requirements supported by measured evidence.

8. Evidence and limitations

This guide reconstructs the systems from retained source code, configuration, submitted reports, deployment screenshots, and the original developer's firsthand recollection. The AWS account is unavailable, so claims about the exact historical runtime configuration are bounded by the retained evidence. Where artifacts conflict, the conflict is recorded instead of silently selecting one version.

8.1 Principal retained evidence

System	Evidence
6.3D	sensor-simulator.js; test-state-validation.js; test-multiple-connections-singleproc.js; documents/flows.json; documents/6.3D.pdf
6.4HD	services/*/server.js; Mongoose models; Docker Compose; Kubernetes Deployments, Services, HPAs, ConfigMaps and Secrets templates; documents/6.4HD.pdf
Shared subscription semantics	AWS IoT Core Developer Guide: https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html

8.2 Claims this guide intentionally does not make

- It does not claim AWS, MQTT, Node-RED, DynamoDB, MongoDB, microservices, or Kubernetes are inherently incorrect choices.
- It does not claim the report screenshots prove every runtime behaviour after the deployment became unavailable.
- It does not recommend production-scale infrastructure for the month-one rework without demonstrated requirements.
- It does not equate a working happy-path demonstration with correctness under retry, restart, concurrency, and scaling.

Final principle

Architecture quality is not measured by how many named technologies appear in the diagram. It is measured by whether responsibilities, state, failure semantics, and operational cost match the system's real requirements.